TP2 — Séance 2 : PPC avec MiniZinc

Cette fiche fait suite à la fiche ***TD2*** *(programmation en nombres entiers & programmation par contraintes — modélisation et résolution à la main) : vous y avez modélisé un réseau de contraintes $N = (X, D, C)$ et tracé l'algorithme de cohérence d'arc sur papier. Ici, vous passez à la pratique : écrire vous-même des modèles* ***MiniZinc*** *– le langage* déclaratif *déjà rencontré au TP1 pour la PL, que vous retrouvez ici dans son usage naturel : la PPC – avec le solveur* ***Gecode*** *(le solveur CP par défaut de MiniZinc, parfaitement adapté ici, contrairement au TP1 où il fallait le remplacer par HiGHS pour des variables continues). Durée indicative : ~1h.*

Fichiers nécessaires (à compléter) :

`Seance2_TP_PPC_minizinc_*_a_completer.mzn`

Pour chaque exercice, complétez le fichier `_a_completer.mzn` *correspondant, puis exécutez-le avec le solveur* ***Gecode*** *:*

```
minizinc -solver Gecode -a Seance2_TP_PPC_minizinc_<nom>_a_completer.mzn
```

(l'option `-a` *énumère* ***toutes*** *les solutions – utile pour vérifier un dénombrement ; omettez-la pour n'obtenir que la première solution).*

Exercice 1 — Les n reines en MiniZinc *(fichier `queens`)*

Question 1 • Dans le fichier `a_completer`, complétez les 3 contraintes `alldifferent` du Modèle 1 (TD2, Exercice 6, Question 1 : lignes, diagonale montante, diagonale descendante).

Question 2 • Exécutez avec `n=4` et l'option `-a`. Combien de solutions trouvez-vous ? Cela correspond-il à ce que vous attendiez ?

Question 3 • Repassez à `n=8` (le problème classique des 8 reines) et relancez avec `-a`. Combien de solutions ?

Exercice 2 — Le Sudoku du cours, en MiniZinc *(fichier `sudoku`)*

Le fichier `a_completer` contient une grille 9×9 partiellement remplie (30 indices donnés) – exactement la grille présentée en cours (section « Le même modèle, à toutes les échelles »), mais avec la plupart des cases vides.

Question 1 • Complétez les 3 familles de contraintes `alldifferent` (lignes, colonnes, blocs 3×3) à partir du modèle `sudoku.mod` rappelé en cours.

Question 2 • Résolvez (sans `-a`, une seule solution suffit) et comparez la grille obtenue à celle du cours. Combien de solutions ce puzzle admet-il au total (relancez avec `-a` pour vérifier) ?

Question 3 • *(Bonus)* Ajoutez la contrainte redondante vue en cours ($\sum = 45$ sur chaque ligne/colonne/bloc). Le résultat change-t-il ? Pourquoi est-ce attendu ?

Exercice 3 Ordonnancement ExedBike en MiniZinc *(fichier `ordonnancement`)*

Même instance qu'au cours (CP-SAT/OR-Tools) : ExedBike doit ordonnancer 3 lots sur une même ligne de production – Urban (durée 3h), Cargo (durée 5h), Sport (durée 4h) – avec un horizon de planification de 15h, en minimisant la date de fin du dernier lot (le *makespan*). En MiniZinc, la contrainte globale correspondant à `NoOverlap` (OR-Tools) est `disjunctive`.

Question 1 • Complétez la contrainte `disjunctive` et l'objectif (minimiser le makespan) dans le fichier `a_completer`.

Question 2 • Résolvez. Quel makespan optimal obtenez-vous ? Correspond-il à la borne inférieure évidente pour ce problème (une seule ligne, donc les lots s'enchaînent nécessairement) ?

Question 3 • En 2-3 phrases : comparez votre expérience de modélisation entre CP-SAT/OR-Tools (`NoOverlap` + `IntervalVar`) et MiniZinc (ici, `disjunctive`) sur ce même problème.

Pour aller plus loin : les trois modèles MiniZinc de cette fiche (reines, Sudoku, ordonnancement) sont volontairement proches de leurs équivalents Python/OR-Tools – comparez les deux formulations côte à côte pour consolider votre compréhension des deux grandes familles de solveurs CP (API impérative type OR-Tools vs langage déclaratif type MiniZinc), un point de méthode utile pour le choix d'outil dans votre futur projet professionnel.